

数据库编程

SQL语言的三种使用方式：

- 交互式SQL (Interactive SQL – ISQL)
- 嵌入式SQL (Embedded SQL – ESQL)
- 过程化SQL (Procedural Language SQL – PL/SQL)

	使用方式	应用场景
交互式SQL	命令行 / 批处理	可独立运行，一般供临时用户操作访问数据库用(即席查询, ad-hoc query)
嵌入式SQL	主语言 + ESQL	数据库应用开发
过程化SQL	SQL编程	• 兼有SQL数据访问和高级程序设计语言的流程控制、简单数值处理功能； • 可在数据库服务器中独立运行； • 常用于编写存储过程、存储函数、触发器。

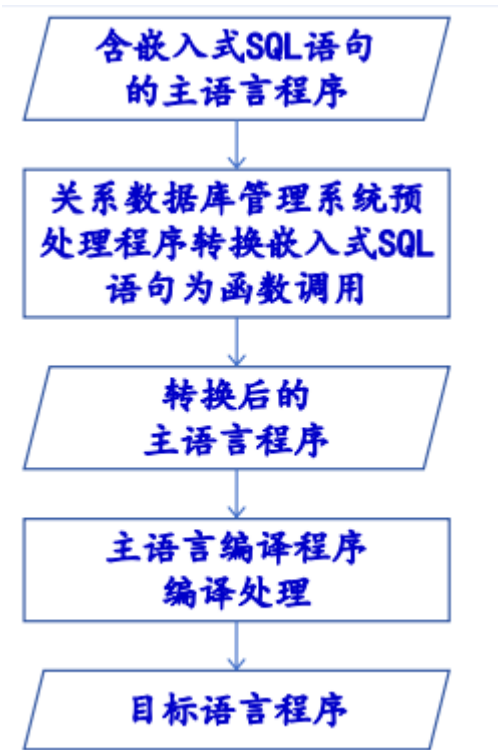
1.嵌入式SQL

为什么要引入嵌入式SQL？

最终用户不再需要书写SQL语句，而是通过直接应用程序来存取访问数据库。

1.1 嵌入式SQL的处理过程

- 主语言：被嵌入的程序设计语言，称为宿主语言，如C，C++等
- 处理过程：预编译方法



使用嵌入式SQL进行数据开发时的问题：

- 主语言语句与嵌入式SQL语句的区别

- 主语言变量和SQL变量的区别
- 主语言程序和嵌入式SQL间的通讯
- SQL查询语句和主语言语句的结果数据交换

数据交换模型：



主语言语句与嵌入式SQL语句的区别：

□ （在C语言中）嵌入式SQL语句

- 带有前缀 **EXEC SQL** 和后缀 **;**
- 使用 **into子句** 来获取结果元组值
(**SELECT ... INTO ...** 或 **FETCH ... INTO ...**)
- 用主变量 **:host_var** 保存查询结果元组中的属性值
通过前缀 **:** 来区分主变量和SQL语言中的表名或属性名（也被称为SQL变量）

```

EXEC SQL select Sno, Sname, Sage
         into :hsno, :hsname, :hsage
         from Student
         where Sno = :givensno ;
  
```

1.2 嵌入式SQL语句和主语言之间的通信

- 向主语言传递SQL语句的执行状态信息，使主语言能够据此控制程序流程，主要用**SQL通信区**实现
- 主语言向SQL语句提供执行参数，主要用**主变量**实现
- 将SQL语句查询数据库的结果交给主语言处理，主要用**主变量和游标**实现

1.2.1 SQL通信区

SQLCA是一个数据结构，用途：

- SQL语句执行后，数据库系统反馈给应用程序的信息
 - 描述当前系统工作状态
 - 描述运行环境
- 上述信息将送到并保存在SQLCA中
- 应用程序从SQLCA中获取信息并执行语句

使用方法：

➤ 定义SQLCA

- 用EXEC SQL INCLUDE SQLCA定义

➤ 使用SQLCA

- SQLCA中有一个用于存放每次执行SQL语句后返回当前SQL语句执行状态的代码变量SQLCODE;
- 如果SQLCODE等于预定义的常量SUCCESS, 则表示SQL语句成功, 否则表示出错;
- 应用程序每执行完一条SQL 语句之后都应该测试一下SQLCODE的值, 以了解该SQL语句执行情况并做相应处理。

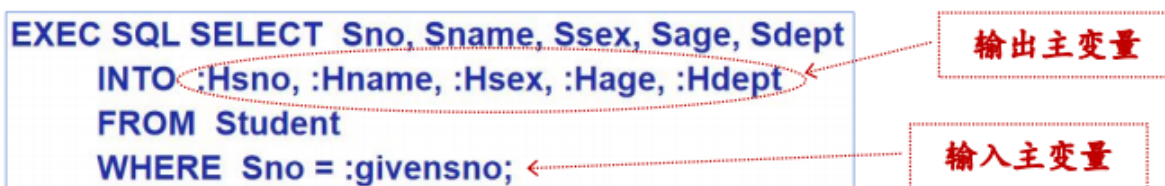
1.2.2 主变量

定义:

- 嵌入式SQL语句中可以使用主语言的程序变量来输入或输出数据
- 在SQL语句中使用的主语言程序变量称为主变量

类型:

- 输入主变量: 由应用程序对其赋值, 在SQL语句引用
- 输出主变量: 由SQL语句对其赋值或设置状态信息, 返回给应用程序



指示变量: 是一个整型变量, 一类特殊的主变量, 用来指示相关主变量的值是否为空值。

- 一个主变量可以附带一个指示变量

指示变量用途:

- 指示输入变量是否为空值
- 检测输出变量是否为空值, 是否被截断

指示变量的取值:

0	为主变量分配了一个数据库值, 而不是空
>0	一个被截断的数据库字符串被分配给了主变量。
-1	数据库值为空, 并且主变量值不是一个有意义的值。

主变量和指示变量的使用方法:

❑ 在SQL语句中使用主变量和指示变量的方法

➢ 声明（定义）主变量和指示变量

BEGIN DECLARE SECTION

..... /* 声明主变量和指示变量，方法与普通主语言变量一样 */

END DECLARE SECTION

➢ 使用主变量

- 声明之后的主变量可以在SQL语句中任何一个能够使用表达式的地方出现
- 为了与数据库对象名（表名、视图名、列名等）区别，SQL语句中的主变量名前要加冒号（:）作为标志

➢ 使用指示变量

- 指示变量前也必须加冒号标志，且必须紧跟在所指主变量之后，用‘空格’或保留字 Indicator 相分隔

❑ 在SQL语句之外(主语言语句中)，可以直接使用主变量和指示变量，不必加冒号

示例：

[例8.3] 指示变量Gradeid，用于指示查询返回的Grade属性值是否为空

```
EXEC SQL SELECT Sno, Cno, Grade
          INTO :Hsno, :Hcno, :Hgrade :Gradeid
          FROM SC
          WHERE Sno = :givensno AND Cno = :givencno ;
```

[例] 在选课关系SC中，根据输入的课程号将该课程的成绩都置为空值（在主变量与指示变量之间用空格或保留字 Indicator 相分隔）

```
gra_ind = -1 ;
EXEC SQL UPDATE SC
          SET Grade = :Hgrade INDICATOR :gra_ind
          WHERE cno = :givencno;
```

主变量和指示变量的声明：

❑ 为了使用主语言变量，必须首先在DECLARE SECTION部分声明这些变量，Why？

```
exec sql begin declare section;
      char hsno[10], givensno[10], hsname[21];
      int hsage;
exec sql end declare section;
.....
EXEC SQL select Sno, Sname, Sage
          into :hsno, :hsname, :hsage
          from Student
          where Sno = :givensno ;
```

❑ [例3.5] 建立“学生”表Student。

```
CREATE TABLE Student (
  Sno CHAR(9) PRIMARY KEY,
  Sname CHAR(20) UNIQUE,
  Ssex CHAR(2),
  Sage SMALLINT,
  Sdept CHAR(20)
);
```

主语言变量声明语句（DECLARE SECTION）作用：

- 在编译时就可以检查主语言变量与其对应属性的数据类型是否一致
- 为接收从数据库返回的结果值而预先申请足够大的内存空间

示例：

```

1  /* 'begin declare' 和 'end declare' 标志着主变量定义的开始与结束，只有在declare
   section 中定义的主语言变量，才能被用在ESQL语句中，实现应用程序与数据库之间的数据交换。*/
2  exec sql begin declare section; /* 主变量声明开始 */
3  /* 字符类型的主变量，需要预先分配好足够大的内存空间（至少比对应的表中属性的最大长度+1，避免
   在返回查询结果时出现内存溢出错误。*/
4  char Deptname[21];
5  char Hsno[10];
6  char Hsname[21];
7  char Hssex[3];
8  int HSage;
9  int NEWAGE;
10 exec sql end declare section; /* 主变量声明结束 */

```

1.2.3 游标 (Cursor)

使用原因：

- SQL语言和主语言具有不同的数据处理方法：非过程化vs面向过程
- 不同的数据处理对象：SQL语言是面向集合的，一条语句可以产生多条记录；主语言是面向记录的

定义：

- 游标是系统为用户开设的一个数据缓冲区，用于存放SQL查询的执行结果
- 每个游标都有一个名字——游标名
- 当游标被打开后，用户可以用SQL语句逐一从游标中获取结果记录，并赋给主变量，交给主变量进行处理

1.2.4 建立和关闭数据库连接

(1) 建立数据库连接

EXEC SQL CONNECT TO target [AS connection-name] [USER user-name];

➤ target是要连接的数据库服务器

- 可以是一个常见的服务器标识串，如 <dbname>@<hostname>:<port>
- 可以是包含服务器标识的SQL串常量
- 也可以是 DEFAULT

➤ connect-name是可选的连接名，连接名必须是一个有效的标识符

➤ 在整个程序内只有一个连接时可以不指定连接名

➤ 程序运行过程中可以修改当前连接

EXEC SQL SET CONNECTION connection-name | DEFAULT;

(2) 关闭数据库连接

EXEC SQL DISCONNECT [connection-name];

1.2.5 异常处理

- whenever语句用于定义异常处理方法
- 最常见的执行异常是：sqlerror和not found
- 最常用的异常处理方法：用goto语句跳转到特定的语句标号处，进行异常处理

语句标号

exec sql whenever sqlerror goto report_error ;

exec sql whenever not found goto notfound ;

Whenever语句：为SQL语句的执行设置一个异常捕获机制

```
1 EXEC SQL WHENEVER condition action
```

预编译器在处理过程中，会根据WHENEVER语句的定义，在每一条嵌入SQL语句之后添加下面的异常状态检查命令：

```
1 if(condition){action}
```

CONDITIONS:

① SQLERROR

- 严重的SQL语句执行错误，出现这类异常系统将终止应用程序的执行。

② NOT FOUND

- SQL语句得到成功执行，但并没有访问到任何元组（Select, Fetch, Insert, Update, or Delete等数据操纵语句）
- 捕捉这类异常，通常用于结束循环（如FOR、WHILE等循环），或者改变程序的控制流程（如用于IF、SWITCH等语句）

③ SQLWARNING

- 非严重但值得注意的异常，通常不会影响到应用程序的执行
- 相关的warning信息被保存在SQL通讯区 **SQLCA** 中；

ACTIONS:

① CONTINUE

- 忽略相关的异常，不改变程序的控制流程
- 也可用于撤消之前关于**condition**的异常处理定义

② GOTO label

- 应用程序将跳转到标号label指定处，继续执行

③ STOP

- 立即终止应用程序的执行，同时回滚(rollback)当前事务并撤消与数据库的连接(disconnect)

④ DO function | BREAK | CONTINUE

- 可以用DO function调用一个C语言函数，函数执行结束后，控制将返回触发该异常的SQL语句之后继续执行；也可以用BREAK或CONTINUE来改变触发该异常的SQL语句所在循环体的执行路径。

1.2.6 程序结构

只考虑数据库访问部分，其程序结构大致如下：

① The Declare Section

- 声明主变量、指示变量、游标

② Condition Handling

- 异常处理规则定义

③ SQL Connect to Database

- 建立数据库连接

④ Main Body of Application Program

- 用主语言编写的应用程序，包括应用界面和数据处理逻辑的实现
- 用ESQL编写的数据库访问语句

⑤ SQL Disconnect

- 撤消与数据库的连接

1.3 不用游标的SQL语句

种类：

- 说明性语句
- 数据定义语句
- 数据控制语句
- 查询结果为单记录的SELECT语句
- 非CURRENT形式的INSERT、DELETE、UPDATE等增删改语句

1.3.1 查询结果为单记录的SELECT语句

这类语句不需要使用游标，只需用INTO子句指定存放查询结果的主变量。

□ [例8.2] 根据学生号码查询学生信息。

```
/* 首先，将要查询的学生的学号赋给主变量givensno，然后执行下述查询 */  
EXEC SQL SELECT Sno, Sname, Ssex, Sage, Sdept  
          INTO :Hsno, :Hname, :Hsex, :Hage, :Hdept  
          FROM Student  
          WHERE Sno = :givensno;  
/* 如果执行成功，结果元组被保存在主变量Hsno、Hname.....中 */
```

□ 说明：

- INTO子句、WHERE子句和HAVING短语的条件表达式中均可以使用主变量
- 查询返回的记录中，可能某些列为空值NULL
- 如果查询结果实际上并不是单条记录，而是多条记录，则程序出错，关系数据库管理系统会在SQLCA中返回错误信息

指示变量的使用：

- [例8.3] 查询某个学生选修某门课程的成绩。假设已经把将要查询的学生的学号赋给了主变量givensno，将课程号赋给了主变量givencno。

```
EXEC SQL SELECT Sno,Cno,Grade
          INTO :Hsno, :Hcno, :Hgrade :Gradeid /* 指示变量 Gradeid */
          FROM SC
          WHERE Sno = :givensno AND Cno = :givencno;

IF (Gradeid < 0) {
    ..... /* 学生givensno在课程givencno上的成绩为空值，数据库系统不会在主
           变量Hgrade中写入任何值 */
} ELSE {
    ..... /* 学生givensno在课程givencno上的成绩不为空，且保存在主变量
           Hgrade中 */
}
```

1.3.2 非CURRENT形式的增删改语句

在UPDATE的SET子句和WHERE子句中可以使用主变量，SET子句还可以使用指示变量。

- [例8.4] 修改某个学生选修1号课程的成绩。

```
/* 需修改成绩的学生的学号已赋给主变量 givensno */
/* 新的成绩已赋给主变量 newgrade */
EXEC SQL UPDATE SC
      SET Grade = :newgrade
      WHERE Sno = :givensno and Cno = '1' ;
```

- [例8.5] 某个学生新选修了某门课程，将有关记录插入SC表中。假设插入的学号已赋给主变量stdno，课程号已赋给主变量couno。

```
/* 由于该学生刚选修课程，成绩应为空，所以要把成绩指示变量
   gradeid赋值为 -1 */
gradeid = -1; /* gradeid为指示变量 */
EXEC SQL INSERT
      INTO SC(Sno, Cno, Grade)
      VALUES(:stdno, :couno, :gr :gradeid);
/* stdno、couno、gr为主变量 */
```

1.4 使用游标的SQL语句

种类：

- 查询结果为多条记录的SELECT语句
- CURRENT形式的UPDATE语句
- CURRENT形式的DELETE语句

当一个查询只会返回单条结果元组，或者不确定该查询是否肯定返回单条结果元组时，也可以使用游标来完成SQL语句。

1.4.1 查询结果为多条记录的SELECT语句

游标机制的本质：在数据交换中，数据库SQL中的变量是**集合型**的而程序设计语言中的主变量则是**标量型**，因此数据库中SQL变量不能直接供程序设计语言使用，而需要有一种机制将SQL变量中的集合量逐个取出后送入应用程序主变量内供其使用，而提供此种机制就是游标。

基于游标的数据交换特点：**一次一行原则**。

游标操作：

1. 声明游标：为某一映像语句的结果集合定义一个命名的游标
2. 打开游标：执行相应的映像语句并打开获得的结果集，此时游标处于活动状态并指向结果集合的第一条记录的前面
3. 推进游标指针并取当前记录：将游标推向结果集合中的下一条记录，读出游标所指向记录的值并赋值给对应的主语言变量
4. 关闭游标：关闭所使用的游标，释放相关的系统资源

1.4.2 CURRENT形式的UPDATE和DELETE语句

非CURRENT形式的UPDATE语句和DELETE语句，即常规的、不使用游标的UPDATE和DELETE操作，其特点是：

- 面向集合的操作
- 一次修改或删除所有满足条件的记录

CURRENT形式的UPDATE语句和DELETE语句的用途：

- 如果只想修改或删除其中某个记录
 - 用带游标的SELECT语句查出所有满足条件的记录；
 - 从中进一步找出要修改或删除的记录；
 - 用CURRENT形式的UPDATE或DELETE语句修改或删除之
 - 在UPDATE语句或DELETE语句中要用子句：WHERE CURRENT OF <游标名>
 - 表示修改或删除最近一次取出的记录，即游标指针当前指向的记录

不能使用CURRENT形式的UPDATE语句和DELETE语句：

- 当游标定义中的SELECT语句带有UNION或ORDER BY子句
- 该SELECT语句相当于定义了一个不可更新的视图

示例：

```

EXEC SQL DECLARE SX CURSOR FOR /*定义游标SX*/
    SELECT Sno,Sname,Ssex,Sage /*游标SX对应的查询语句*/
    FROM Student
    WHERE SDept = :deptname;

.....
EXEC SQL OPEN SX; /*打开游标SX, 执行游标对应的查询, 并指向查询结果的第一行*/
for ( ;; ) /*用循环结构逐条处理结果集中的记录*/
{
    EXEC SQL FETCH SX INTO :HSno,:Hsname,:HSsex,:HSage;
        /*推进游标, 将当前数据放入主变量*/
    if (SQLCA.SQLCODE != 0) /* SQLCODE != 0, 表示操作不成功 */
        break; /* 利用SQLCA中的状态信息决定何时退出循环 */

    .....
    EXEC SQL UPDATE Student /* 嵌入式SQL更新语句 */
        SET Sage = :NEWAGE
        WHERE CURRENT OF SX; /*对当前游标指向的学生年龄进行更新*/
    .....
}
EXEC SQL CLOSE SX; /*关闭游标SX, 不再和查询结果对应*/

```

1.5 动态SQL

静态SQLvs动态SQL:

	描述	优点	缺点
静态SQL	在应用开发阶段就能够确定下来的SQL语句。	在预编译时, DBMS将对SQL语句进行分析和执行计划优化, 可确保数据库访问的正确性。	只能根据缺省参数值进行优化, 其访问路径并非是最优路径。
动态SQL	在运行时动态生成的SQL语句。	可以根据运行时数据库的当前状态选择最优访问路径。	动态地进行SQL语句的语法分析和访问路径选择, 需要额外的执行开销。

1.5.1 使用SQL语句主变量

程序主变量包含的内容是SQL语句的内容, 而不是原来保存数据的输入或输出变量。

SQL语句主变量在程序执行期间可以设定不同的SQL语句。

□ [例8.6] 创建基本表TEST。

```

EXEC SQL BEGIN DECLARE SECTION;
    const char *stmt = "CREATE TABLE test(a int);" ;
        /* SQL语句主变量, 内容是创建表的SQL语句 */
EXEC SQL END DECLARE SECTION;

.....
EXEC SQL EXECUTE IMMEDIATE :stmt; /*执行动态SQL语句*/

```

1.5.2 动态参数

动态参数是SQL语句中的可变元素, 使用参数符号 (?)表示该位置的数据在运行时设定。

和主变量区别:

- 动态参数的输入不是编译时完成绑定
- 而是通过PREPARE语句准备主变量和执行语句EXECUTE绑定数据或主变量来完成

步骤:

- 声明SQL语句主变量
- 准备SQL语句 (PREPARE)

EXEC SQL PREPARE <语句名> FROM <SQL语句主变量>

1.5.3 执行准备好的语句

EXEC SQL EXECUTE <语句名>
[INTO <主变量表>]
[USING <主变量或常量>];

□ [例8.7] 向TEST中插入元组。

```
EXEC SQL BEGIN DECLARE SECTION;
    const char *stmt = "INSERT INTO test VALUES(?);" ;
    /*声明SQL主变量内容是INSERT语句 */
EXEC SQL END DECLARE SECTION;

...
EXEC SQL PREPARE mystmt FROM :stmt; /*准备语句*/

...
EXEC SQL EXECUTE mystmt USING 100;
    /*执行INSERT语句，设定参数? 值为 100 */
EXEC SQL EXECUTE mystmt USING 200;
    /* 执行INSERT语句，设定参数? 值为 200 */
```

2.过程化SQL

为了能够在数据库实现部分应用程序逻辑，需要在交互式SQL语言的基础上扩充部分过程式程序设计语言的成分，称为过程化SQL。

过程化SQL作用：可用于定义触发器、存储过程、存储函数对应的SQL程序块。

过程化SQLvs嵌入式SQL：

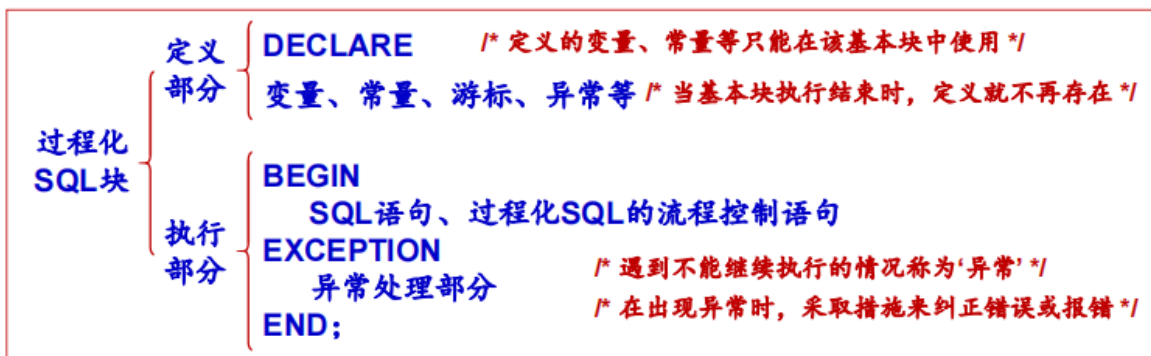
- 可独立编程，不再区分主变量和SQL变量
- 不需要经历从预编译到编译的处理
- 在数据库服务器内部实现数据交换和处理

最常用的过程化SQL：PL/SQL和T-SQL

2.1 过程化SQL的块结构

□ 过程化SQL

- 是对SQL的扩展，增加了过程化语句功能
- 过程化SQL的基本结构是‘块’（图8.2）
 - 块之间可以互相嵌套
 - 每个块完成一个逻辑操作



2.2 变量和常量的定义

2.3 流程控制

3.存储过程和函数

存储过程：

用过程化SQL编写的过程，经编译和优化后存储在数据库服务器中，因而被称为“存储过程”。

- 可以被应用程序或其他存储过程调用。

函数：

函数的定义与存储过程类似，也称为存储函数。

- 与存储过程不同的是函数必须定义返回值的类型

两者区别：

- 使用场景不同：
 - 存储过程通常用于复杂业务逻辑的实现和完成数据库运维管理任务
 - 函数一般用于计算和数据查询任务
- 结果返回方式不同
 - 存储过程本身没有返回值，只能通过访问存储过程的输出参数或输入/输出参数来获取执行结果
 - 函数有返回值类型，把执行结果返回给调用者
- 执行方式不同：
 - 存储过程只能被直接调用
 - 函数可以出现在SQL语句和表达式中

优点：

- 运行效率高
- 降低客户机和数据库服务器之间的通讯开销
- 方便用户业务逻辑的实现和管理

□ 过程化SQL块类型

□ 命名块

- 编译后保存在数据库中，可以被反复调用，运行速度较快。
- 过程和函数是命名块

□ 匿名块

- 每次执行时都要进行编译，它不能被存储到数据库中，也不能在其他过程化SQL块中被调用。

```
DECLARE
..... /* 定义声明部分 */
BEGIN
.....
EXCEPTION
..... /* 异常处理部分 */
END
```

/* 执行部分 */

4.基于驱动的数据库编程

JAVA世界：JDBC

微软世界：ODBC,OLE DB

Python世界：Python DB-API + 各数据库模块（如PyMySQL,SQLite3等）

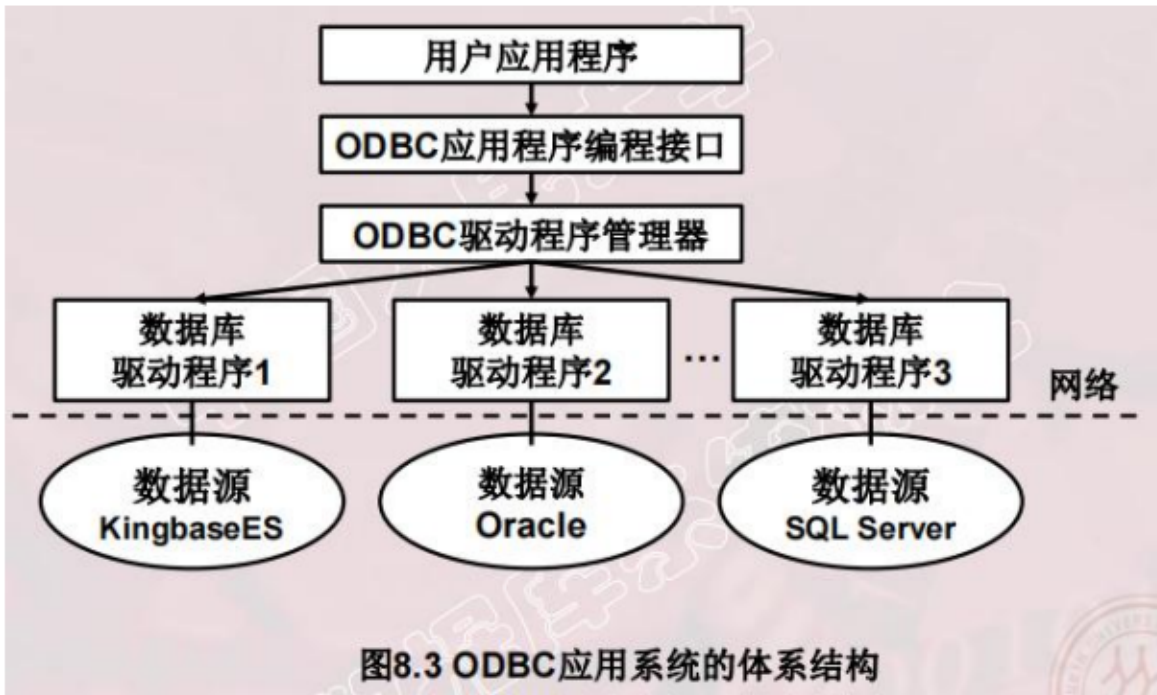
数据交换的流程：

- 连接阶段
- 数据交换阶段
- 断开连接阶段

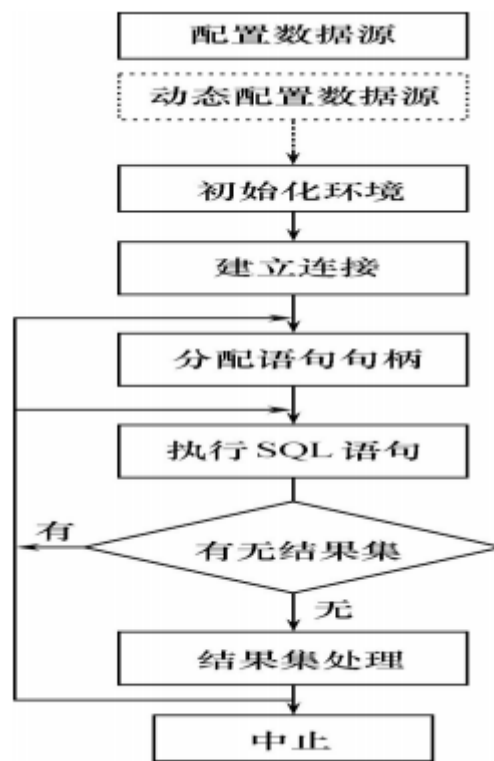
4.1 ODBC 和 OLE DB

面向ODBC驱动的数据交换流程		
阶段	任务	ODBC主要函数
连接阶段	负责资源分配和建立连接	AllocEnv 分配SQL环境 AllocStmt 分配SQL语句 AllocHandle 分配SQL资源 AllocConnect 分配SQL连接 Connect 创建连接
数据交换阶段	数据访问与交换	有关游标的函数 有关诊断的函数 有关动态SQL的函数 有关数据获取的函数 其它函数
断开连接阶段	断开连接并释放资源	SQLFreeStmt 释放语句句柄 SQLDisconnect 撤销与数据库的连接 SQLFreeConnect 释放数据库连接句柄 SQLFreeEnv 释放环境句柄

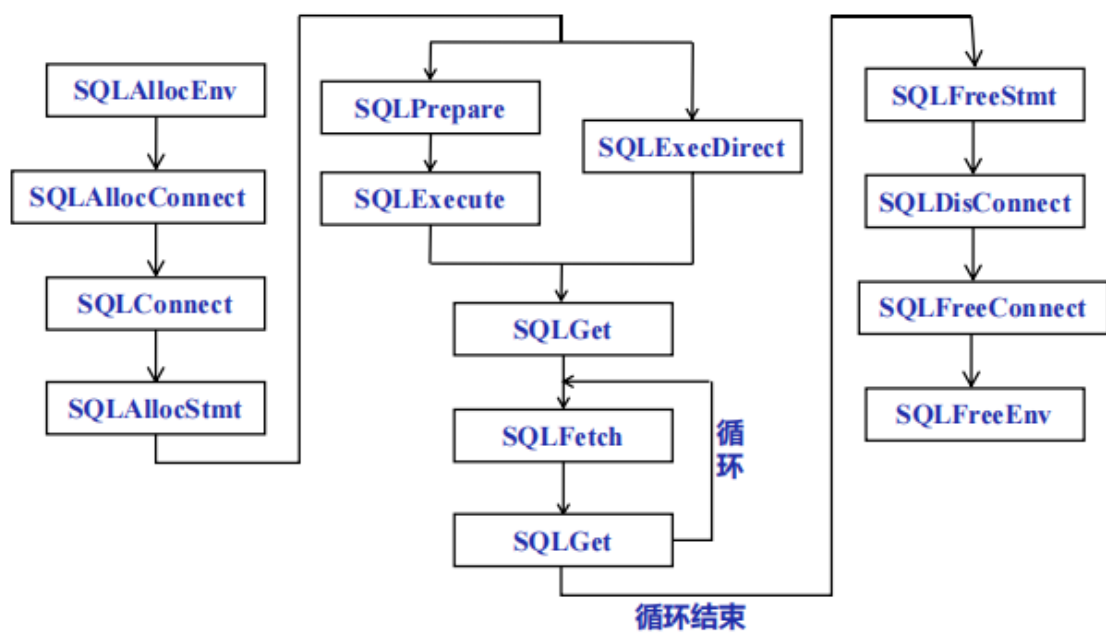
ODBC应用系统的体系架构：



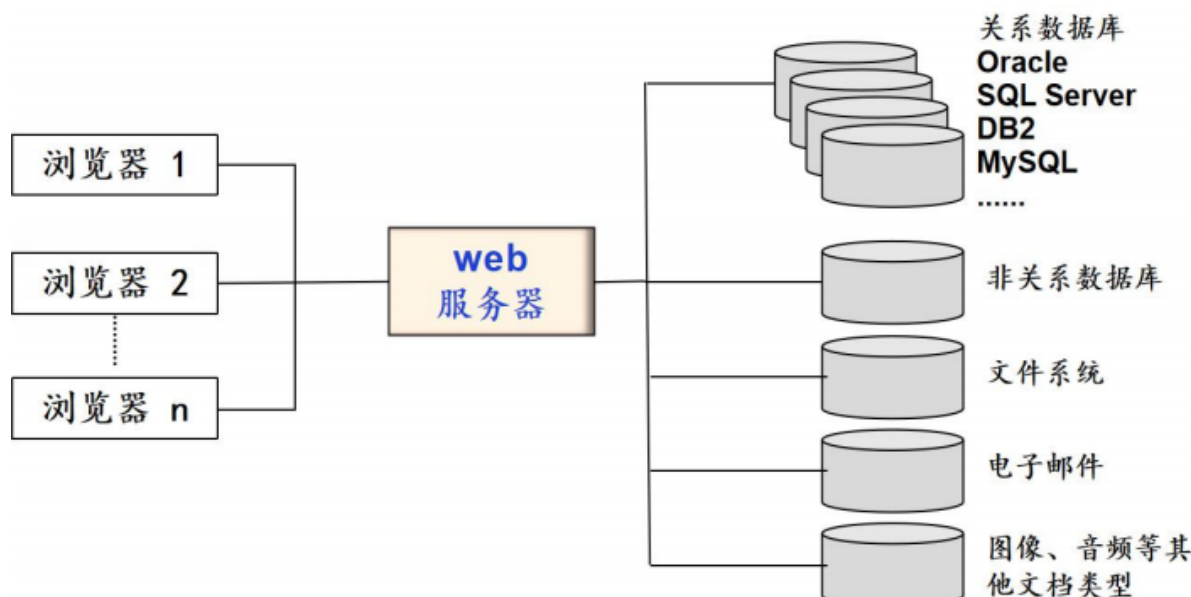
ODBC的工作流程：



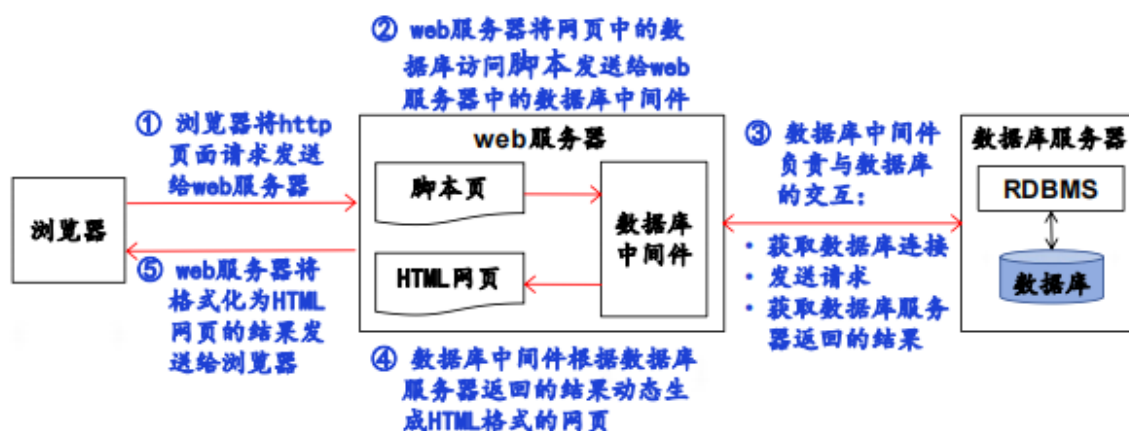
使用ODBC的程序流程：



web数据库应用中的各种数据源：



web数据库应用中的数据交换流程：

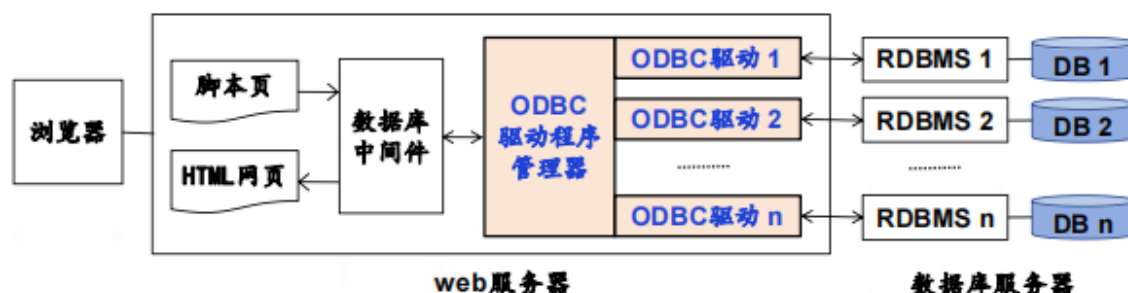


web服务器中的数据库中间件与数据库服务器的交互（互操作）可以通过两者途径来实现：

- 使用DBMS厂商提供的专用数据库访问中间件（如Oracle数据库中的OCI、DB2 CLI、MySQL PDO等）
- 使用通用的数据库访问中间件(如ODBC、OLE DB、JDBC等)

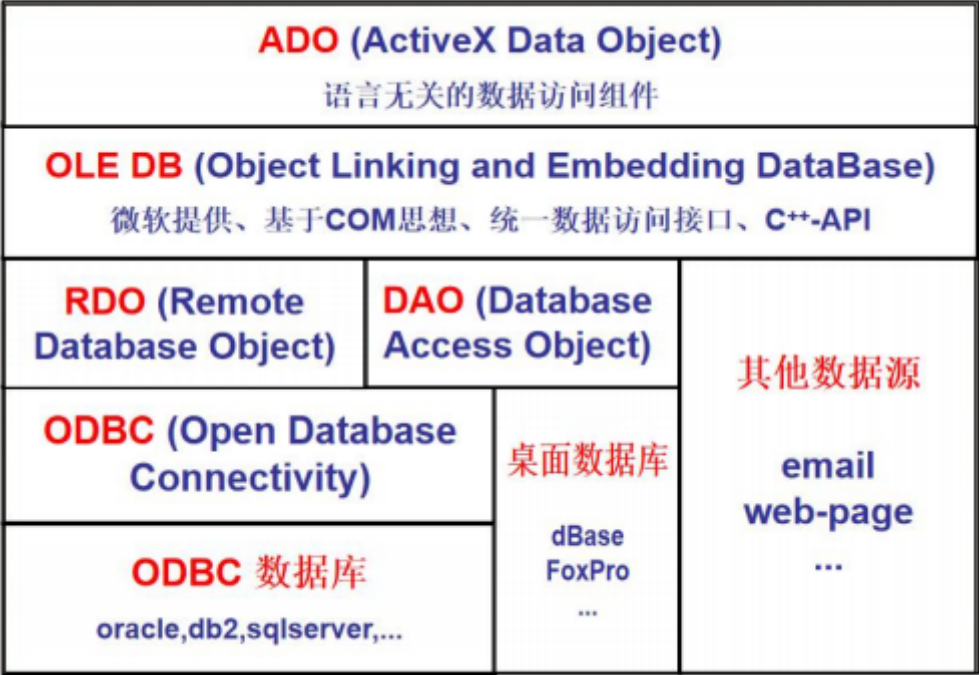
ODBC体系结构：

- 开放式数据库连接（Open Database Connectivity）是数据库服务器的一个标准协议，它向访问数据库的应用程序提供了一种通用的应用程序访问接口，应用程序开发人员不必知道所连接的数据库平台类型，就可以用标准的SQL语言访问数据库中的数据
- ODBC通过ODBC驱动程序管理器将用户调用的SQL语句转换成特定数据库的访问函数，不同数据库提供不同的ODBC驱动程序实现
- 在ODBC客户端，应根据后台数据库类型来选择对应的数据源名称，并指定与后台数据库服务器的连接方式等信息



web方式下的数据库网络访问接口：

- ODBC (Open Database Connectivity): 基于SQL语言的一种标准化的数据库访问API，统一了不同厂商的关系数据库管理系统的访问接口。
- DAO (Database Access Object): 第一个面向对象的数据库访问接口，既可以访问本地的单机桌面数据库，也可以通过ODBC访问远程数据库。
- RDO (Remote Database Object): 封装了ODBC API的对象层，具有比DAO更高的性能，比ODBC更易用
- OLE DB (Object Linking and Embedding DataBase): 微软提出的基于COM思想的一种技术标准，目的是提供一种统一的数据访问接口（包括关系或非关系型的数据源）
- ADO (ActiveX Data Object)



4.2 JDBC编程

一种可以执行SQL语句的JAVA API

